

Java 17 features: A comparison between versions 8 and 17. What has changed over the years?

Disclaimer: this article was originally published on October 22, 2021. However, in December 2022 it was updated with new information regarding the Oracle Enterprise Performance Pack for Java 8. Also, while Java 17 brought a multitude of useful and interesting additions, Oracle published a new LTS release – version 21. Check out our article about [Java 21 features](#) if you want to know what it brings to the table.

In March 2022 Java 8 lost its Oracle Premier Support. It doesn't mean that it won't receive any new updates, but Oracle's effort put into maintaining it will likely be significantly smaller than it is right now.

That means there's a good reason to make the move to a new version. Especially since on September 14th, 2021, Java 17 was released. It's the new Long Term Support version, with Oracle Premier Support to last until [September 2026](#) (at least). What does Java 17 bring? How difficult will the migration be? Is it worth it? I'll try to answer those questions in this article.

It is also worth noting that Java 8 is still getting some expansions – though only for Oracle Java and its costly Java SE Subscription. An Oracle Enterprise Performance Pack for Java 8 was released on July 19, 2022. The version number is 8u345-PERF-b31 and its full release notes can be found [here](#). The additions to this version will be mentioned later in the article when particular features of Java 8 and 17 are compared.

If you're interested in newer versions of Java, my colleague Arek Rosłonec wrote two articles on [Java 21 features](#) and [Java 23 features](#), respectively.

The popularity of Java 8 – a little bit of history



Let's take a look at some of the features.

Java 8, which was released in March 2014, is [currently used by 69% of programmers](#) in their main application. Why is it, after more than 7 years, still the most commonly used version? There are many reasons for that.

Java 8 provided lots of language features that made [Java Developers](#) want to switch from previous versions. Lambdas, streams, functional programming, extensive API extensions – not to mention MetaSpace or G1 extensions. It was *the* Java version to use.

Java 9 appeared 3 years later, in September 2017, and for a typical developer, it changed next to nothing. A new HTTP client, process API, minor diamond operator and try-with-resources improvements.

Sure, Java 9 did bring one significant change, groundbreaking even – [the Jigsaw Project](#). It changed a lot, a great lot of things – but *internally*. Java modularization gives great possibilities, solves lots of technical problems, and applies to everyone, but only a relatively small group of users actually needed to deeply understand the changes. Due to the changes introduced with the Jigsaw Project lots of libraries required additional modifications, new versions were released, some of them did not work properly.

Java 9 migration – in particular for large, corporate applications – was often difficult, time-consuming, and caused regression problems. So why do it, if there is little to gain and it costs a lot of time and money?

Java Development Kit 17 (JDK 17) was released In October 2021. Is it a good time to move on from the 8-year-old Java 8? First, let's see what's in Java 17. What does it bring to the programmer and admin or SRE, when compared to Java 8?

Java 17 vs Java 8 – the changes

This article covers only the changes that I deemed important enough or interesting enough to mention. They are not everything that was changed, improved, optimized in all the years of Java evolution. If you want to see a full list of changes to JDK, you should know that they are tracked as JEPs (JDK Enhancement Proposals). The list can be found in [JEP-0](#).

Also, if you want to compare Java APIs between versions, there is a great tool called [Java Version Almanac](#). There were many useful, small additions to Java APIs, and checking this website is likely the best option if someone wants to learn about all these changes.

As for now, let's analyze the changes and new features in each iteration of Java, that are most important from the perspective of most of us Java Developers.

New var keyword

A new *var* keyword was added that allows local variables to be declared in a more concise manner. Consider this code:

```
// java 8 way
Map<String, List<MyDtoType>> myMap = new HashMap<String, List<MyDtoType>>();
List<MyDomainObjectWithLongName> myList = aDelegate.fetchDomainObjects();

// java 10 way
var myMap = new HashMap<String, List<MyDtoType>>();
var myList = aDelegate.fetchDomainObjects()
```

When using *var*, the declaration it's much, much shorter and, perhaps, a bit more readable than before. One must make sure to take the readability into account first, so in some cases, it may be wrong to hide the type from the programmer. Take care to name the variables properly.

Unfortunately, it is not possible to assign a lambda to a variable using *var* keyword:

```
// causes compilation error:
//   method reference needs an explicit target-type
var fun = MyObject::mySpecialFunction;
```

It is, however, possible to use the *var* in lambda expressions. Take a look at the example below:

```
boolean isThereAneedle = stringsList.stream()
    .anyMatch((@NonNull var s) -> s.equals("needle"));
```

Using *var* in lambda arguments, we can add annotations to the arguments.

Records

One may say Records are Java's answer to Lombok. At least partly, that is. Record is a type designed to store some data. Let me quote a fragment of [JEP 395](#) that describes it well:

[...] a record acquires many standard members automatically:

- *A private final field for each component of the state description;*
- *A public read accessor method for each component of the state description, with the same name and type as the component;*
- *A public constructor, whose signature is the same as the state description, which initializes each field from the corresponding argument;*
- *Implementations of equals and hashCode that say two records are equal if they are of the same type and contain the same state; and*
- *An implementation of toString that includes the string representation of all the record components, with their names.*

In other words, it's roughly equivalent to Lombok's @Value. In terms of language, it's kind of similar to an enum. However, instead of declaring possible values, you declare the fields. Java generates some code based on that declaration and is capable of handling it in a better, optimized way. Like enum, it can't extend or be extended by other classes, but it can implement an interface and have static fields and methods. Contrary to an enum, a record can be instantiated with the new keyword.

A record may look like this:

```
record BankAccount (String bankName, String accountNumber) implements
HasAccountNumber {}
```

And this is it. Pretty short. Short is good!

Any automatically generated methods can be declared manually by the programmer. A set of constructors can be also declared. Moreover, in constructors, all fields that are *definitely unassigned* are implicitly assigned to their corresponding constructor parameters. It means that the assignment can be skipped entirely in the constructor!

```
record BankAccount (String bankName, String accountNumber) implements
HasAccountNumber {
    public BankAccount { // <-- this is the constructor! no () !
        if (accountNumber == null || accountNumber.length() != 26) {
            throw new ValidationException("Account number invalid");
        }
        // no assignment necessary here!
    }
}
```

For all the details like formal grammar, notes on usage and implementation, make sure to consult the [JEP 359](#). You could also check StackOverflow for the most upvoted [questions on Java Records](#).

Extended switch expressions

Switch is present in a lot of languages, but over the years it got less and less useful because of the limitations it had. Other parts of Java grew, *switch* did not. Nowadays *switch* cases can be grouped much more easily and in a more readable manner (note there's no *break*!) and the *switch* expression itself actually returns a result.

```
DayOfWeek dayOfWeek = LocalDate.now().getDayOfWeek();
boolean freeDay = switch (dayOfWeek) {
    case MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY -> false;
    case SATURDAY, SUNDAY -> true;
};
```

Even more can be achieved with the new *yield* keyword that allows returning a value from inside a code block. It's virtually a *return* that works from inside a case block and sets that value as a result of its *switch*. It can also accept an expression instead of a single value. Let's take a look at an example:

```
DayOfWeek dayOfWeek = LocalDate.now().getDayOfWeek();
boolean freeDay = switch (dayOfWeek) {
    case MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY -> {
```

```

        System.out.println("Work work work");
        yield false;
    }
    case SATURDAY, SUNDAY -> {
        System.out.println("Yey, a free day!");
        yield true;
    }
};

```

Instanceof pattern matching

While not a groundbreaking change, in my opinion, *instanceof* solves one of the more irritating problems with the Java language. Did you ever have to use such syntax?

```

if (obj instanceof MyObject) {
    MyObject myObject = (MyObject) obj;
    // ... further logic
}

```

Now, you won't have to. Java can now create a local variable inside the *if*, like this:

```

if (obj instanceof MyObject myObject) {
    // ... the same logic
}

```

It is just one line removed, but it was a totally unnecessary line in terms of the code flow. Moreover, the declared variable can be used in the same *if* condition, like this:

```

if (obj instanceof MyObject myObject && myObject.isValid()) {
    // ... the same logic
}

```

[post_frame text="Schedule a call" link="#contact"]Are you looking for Java developers?
[/post_frame]

Sealed classes

This is a tricky one to explain. Let's start with this – did the “no default” warning in *switch* ever annoy you? You covered all the options that the domain accepted, but still, the warning was there. Sealed classes let you get rid of such a warning for the *instanceof* type checks.

If you have a hierarchy like this:

```
public abstract sealed class Animal
    permits Dog, Cat {
}
public final class Dog extends Animal {
}
public final class Cat extends Animal {
}
```

You will now be able to do this:

```
if (animal instanceof Dog d) {
    return d.woof();
}
else if (animal instanceof Cat c) {
    return c.meow();
}
```

And you won't get a warning. Well, let me rephrase that: if you get a warning with a similar sequence, that warning will be *meaningful*! And more information is always good.

I have mixed feelings about this change. Introducing a cyclic reference does not seem like a good practice. If I used this in my production code, I'd do my best to hide it somewhere deep and never show it to the outside world – I mean, never expose it through an API, not that I would be ashamed of using it in a valid situation.

TextBlocks

Declaring long strings does not often happen in Java programming, but when it does, it is tiresome and confusing. Java 13 came up with a fix for that, further improved in later releases.

A multiline text block can now be declared as follows:

```
String myWallOfText = """
_____      -   -
|  _  \      | | ( )
| |_/ /  _  _| | _ _ _ _ _
|  _/ ' _/ _ \ _| | | | / _|
| | | | |  _/ | | | | \ _ \
\_|  | _ \ _| \_| \_| \_| _/

"""
```

There is no need for escaping quotes or newlines. It is possible to escape a newline and keep the string a one-liner, like this:

```
String myPoem = """
Roses are red, violets are blue - \
Pretius makes the best software, that is always true
"""
```

Which is the equivalent of:

```
String myPoem = "Roses are red, violets are blue - Pretius makes the best
software, that still is true".
```

Text blocks can be used to keep a reasonably readable json or xml template in your code. External files are still likely a better idea, but it's still a nice option to do it in pure Java if necessary.

Better NullPointerExceptions

So, I had this chain of calls in my app once. And I think it may look familiar to you too:

```
company.getOwner().getAddress().getCity();
```

And I got an NPE that told me precisely in which line the null was encountered. Yes, it was that line. Without a debugger I couldn't tell which object was null, or rather, which invoke operation has actually caused the problem. Now the message will be specific and it'll tell us that the JVM "cannot invoke Person.getAddress()".

Actually, this is more of a JVM change than a Java one – as the bytecode analysis to build the detailed message is performed at runtime JVM – but it does appeal to programmers a lot.

New HttpClient

There are many libraries that do the same thing, but it is nice to have a proper HTTP client in Java itself. You can find a nice introduction to the new APIs in [Baeldung](#).

New Optional.orElseThrow() method

A *get()* method on Optional is used to get the value under the Optional. If there is no value, this method throws an exception. Like in the code below.

```
MyObject myObject = myList.stream()
    .filter(MyObject::someBoolean)
    .filter((b) -> false)
    .findFirst()
    .get();
```

Java 10 introduced a new method in Optional, called *orElseThrow()*. What does it do? Exactly the same! But consider the readability change for the programmer.

```
MyObject myObject = myList.stream()
    .filter(MyObject::someBoolean)
    .filter((b) -> false)
    .findFirst()
    .orElseThrow();
```

Now, the programmer knows exactly what will happen when the object is not found. In fact, using this method is recommended instead of the simple – albeit ubiquitous – *get()*.

Other small but nice API changes

Talk is cheap, this is the code. In bold are the new things.

```
// invert a Predicate, will be even shorter with static import
collection.stream()
    .filter(Predicate.not(MyObject::isEmpty))
    .collect(Collectors.toList());
// String got some new stuff too
“\nPretius\n rules\n all!”.repeat(10).lines().
    .filter(Predictions.not(String::isBlank))
    .map(String::strip)
    .map(s -> s.indent(2))
    .collect(Collectors.toList());
// no need to have an instance of array passed as an argument
String[] myArray= aList.toArray(String[]::new);
// read and write to files quickly!
// remember to catch all the possible exceptions though
Path path = Files.writeString(myFile, "Pretius Rules All !");
String fileContent = Files.readString(path);
// .toList() on a stream()
String[] arr={"a", "b", "c"};
var list = Arrays.stream(arr).toList();
```

JVM 17 vs JVM 8 changes

Project Jigsaw



Project Jigsaw turned some things on their heads, but now everything works well.

JDK 9's Project [Jigsaw](#) significantly altered the internals of JVM. It changed both [JLS](#) and [JVMS](#), added several JEPs (list available in the Project Jigsaw link above), and, most importantly, introduced some breaking changes, alterations that were incompatible with previous Java versions.

[Java 9 modules](#) were introduced, as an additional, highest level of jar and class organization. There's lots of introductory content on this topic, like this one on [Baeldung](#) or these slides from [Yuichi Sakuraba](#).

The gains were significant, though not visible to the naked eye. So-called [JAR hell](#) is no more (have you been there? I was... and it was really a hell), though a [module hell](#) is now a possibility.

From the point of view of a typical programmer, these changes are now almost invisible. Only the biggest and the most complex projects may somehow be impacted. New versions of virtually all commonly used libraries adhere to the new rules and take them into account internally.

Garbage Collectors

As of Java 9, the G1 is the default garbage collector. It reduces the pause times in comparison with the Parallel GC, though it may have lower throughput overall. It has undergone some changes since it was made default, including the ability to return unused committed memory to the OS ([JEP 346](#)).

A ZGC garbage collector has been introduced in Java 11 and has reached product state in Java 15 ([JEP 377](#)). It aims to reduce the pauses even further. As of Java 13, it's also capable of returning unused committed memory to the OS ([JEP 351](#)).

A Shenandoah GC has been introduced in JDK 14 and has reached product state in Java 15 ([JEP 379](#)). It aims to keep the pause times low and independent of the heap size.

Note that in Java 8 you had much fewer options, and if you did not change your GC manually, you still used the Parallel GC. Simply switching to Java 17 may cause your application to work faster and have more consistent method run times. Switching to, then unavailable, ZGC or Shenandoah may give even better results.

Finally, there's a new No-Op Garbage Collector available ([JEP 318](#)), though it's an experimental feature. This garbage collector does not actually do any work – thus allowing you to precisely measure your application's memory usage. Useful, if you want to keep your memory operations throughput as low as possible.

If you want to learn more about available options, I recommend reading a great series of articles by [Marko Topolnik](#) that compares the GCs.

The aforementioned G1 garbage collector was added to Oracle Java 8 in the Oracle Enterprise Performance Pack version 8u345. Along with [Compact Strings](#), it can have a significant impact on memory consumption of a Java application.

Container awareness

In case you didn't know, there was a time that Java was unaware that it was running in a container. It didn't take into account the memory restrictions of a container and read available system memory instead. So, when you had a machine with 16 GB of RAM, set your container's max memory to 1 GB, and had a Java application running on it, then often the application would fail as it would try to allocate more memory than was available on the container. A nice [article from Carlos Sanchez](#) explains this in more detail.

These problems are in the past now. As of Java 10, the [container integration](#) is enabled by default. However, this may not be a noticeable improvement for you, as the same change was introduced in Java 8 update 131, though it required enabling experimental options and using `-XX:+UseCGroupMemoryLimitForHeap`.

PS: It's often a good idea to specify the max memory for Java using an `-Xmx` parameter. The problem does not appear in such cases.

CDS Archives

In an effort to make the JVM start faster, the CDS Archives have undergone some changes in the time that passed since the Java 8 release. Starting from JDK 12, creating CDS Archives during the build process is enabled by default ([JEP 341](#)). An enhancement in JDK 13 ([JEP 350](#)) allowed the archives to be updated after each application run.

Class Data Sharing was also implemented for Java 8 in the Oracle Enterprise Performance Pack version 8u345. It is, however, unclear how significant these changes are; the description would suggest that only the scope of [JEP 310](#) was added. I was, however, unable to confirm this.

A great [article from Nicolai Parlog](#) demonstrates how to use this feature to improve startup time for your application.

Java Flight Recorder and Java Mission Control

Java Flight Recorder ([JEP 328](#)) allows monitoring and profiling of a running Java application at a low (target 1%) performance cost. [Java Mission Control](#) allows ingesting and visualizing JFR data. See [Baeldung's tutorial](#) to get a general idea of how to use it and what one can get from it.

Should you migrate from Java 8 to Java 17?

To keep it short: yes, you should. If you have a large, high-load enterprise application and still use Java 8, you will definitely see better performance, faster startup time, lower memory footprint after migrating. Programmers working on that application should also be happier, as there are many improvements to the language itself.

The cost of doing so, however, is difficult to estimate and varies greatly depending on used application servers, libraries, and the complexity of the application itself (or rather the number of low-level features it uses/reimplements).

If your applications are microservices, it's likely that all you will need to do is to change the base docker image to 17-alpine, code version in maven to 17, and everything will work just fine. Some [frameworks or library updates](#) may come in handy (but you're doing them periodically anyway, right?).

All popular servers and frameworks have the Java 9's Jigsaw Project support by now. It's production-grade, it has been heavily tested, and bugfixed over the years. Many products offer migration guides or at least extensive release notes for the Java 9-compatible version.

See a nice [article from OSGI](#) or some [release notes for Wildfly 15](#) mentioning modules support.

If you use Spring Boot as your framework, there are some articles available with migration tips, like [this one](#) in the spring-boot wiki, [this one](#) on Baeldung, and yet [another one](#) on DZone. There's also an [interesting case study from infoq](#). Migrating Spring Boot 1 to Spring Boot 2 is a different topic, it might be worth considering too. There's a [tutorial from Spring Boot](#) itself, and an [article on Baeldung](#) covering this topic.

If your application didn't have custom classloaders, didn't heavily rely on Unsafe, lots of sun.misc or sun.security usages – you're likely to be fine. Consult [this article from JDEP](#) on Java Dependency Analysis Tool, for some changes you may have to make.

Some things were removed from Java since version 8, including Nashorn JS Engine, Pack200 APIs and Tools, Solaris/Sparc ports, AOT and JIT compilers, Java EE, and Corba modules. Some things still remain but are deprecated for removal, like Applet API or Security Manager. And since there are good reasons for their removal, you should reconsider their use in your application anyway.

I asked our Project technical Leaders at Pretius about their experiences with Java 8 to Java 9+ migrations. There were several examples and none were problematic. Here, a library did not work and had to be updated; there, some additional library or configuration was required but overall, it wasn't a bad experience at all.

Conclusion

Java 17 LTS is going to be supported for years to come. On the other hand, Java 8's support has ended. It's certainly a solid reason to consider moving to the new version of Java. In this article, I covered the most important language and JVM changes between versions 8 and 17 (including some information about the Java 8 to Java 9+ migration process), so that it's easier to understand the differences between them – as well as to assess the risks and gains of migration.

If you happen to be a decision maker in your company, the question to ask yourself is this: will there *ever* be "a good time" to leave Java 8 behind? Some money will always have to be spent, some time will always have to be consumed and the risk of some additional work that needs to be done will always exist. If there's never "a good time", now is likely as good of a moment, as there's ever going to be.

Do you need a Java development company?

Pretius has a great team of Java developers, and a lot of experience with using the technology in enterprise-grade systems. We also know our way around many different industries. Do you need Java-based software? Drop us a line at hello@pretius.com (or use the contact form below). We'll get back to you in 48 hours and tell you what we can do for you. You can also check out my other articles on the Pretius blog:

1. [JVM Kubernetes: Optimizing Kubernetes for Java Developers](#)
2. [Project Valhalla – Java on the path to better performance](#)
3. [Clean-architecture Java: How ArchUnit can help your application](#)

Java 17 features FAQ

When was Java 8 released?

Java 8 was released in March 2014.

When was Java 17 released?

Java 17 was released on September 15, 2021.

What is the latest version of Java?

The latest version of Java is Java 20, released in March 2023. However, the latest version with long-term support (LTS) is Java 17.

What version of Java do I have?

You can check your current Java version in the About section in the General tab of the Java Control Panel. You can also type the following command in your bash/cmd:

```
java -version
```

What is Java 17?

It's the latest version of the Java SE platform with long-term support.

How to update to Java 17?

Installing the software is a simple matter of running the executable, but making your system ready for the change can be more complicated.

What are the new features in JDK 17?

JDK 17 is a big Java update with plenty of improvements and new things. It offers the following new features.

1. Enhanced pseudo-Random Number Generators
2. Restore Always-Strict Floating-Point Semantics
3. macOS/AArch64 Port
4. New macOS Rendering pipelines
5. Strongly Encapsulated JDK Internals
6. Deprecate the Applet API for Removal
7. Pattern matching for Switch(Preview)
8. Sealed Classes
9. Removal RMI Activation
10. Deprecate the Security manager for Removal
11. Foreign Functions & memory API(Incubator)
12. Removal Experimental AOT and JIT Compiler
13. Context-Specific Deserialization Filters
14. Vector API(Second Incubator)

What is the difference between Java 17 and Java 18

Java 17 is a long-term support version – it'll be supported for at least eight years. Java 18, on the other hand, will only be a smaller update with some additional features, and 6-months-long support.

Is JRE included in JDK 17?

Yes, as with all JDK versions, JDK 17 includes the Java 17 JRE.

What is the Enterprise Performance Pack for Java 8?

It's a paid subscription that you can buy to get some Java 17 features – such as the G1 garbage collector – in Java 8.